A Brief History of Programming Languages

Machine Languages, Assembly Languages, and High-Level Languages

I. Machine Languages (The First Generation)

A. The first languages developed

B. Even today, still the only language "understood" by computers!

C. Instructions (aka: "statements") in a machine language use only 0's and 1's

(Naturally, this makes machine languages difficult for humans, although not for computers)

D. Because the instructions are based upon the architecture of the computer, each model computer has its own, unique machine language

(This means that machine language programs are not portable among different kinds of computers)

E. Because the operations that a computer can actually do are very limited, and because each machine language instruction specifies exactly one machine-level operation, even very simple programs are quite lengthy.

F. Disadvantages of machine languages (summary)
1. Difficult for humans
2. Programs are not portable
3. Programs are quite lengthy

G. Typical machine language code:

0101 0011 1110 1000
0110 0011 1110 1010
0111 0011 1110 1100

The first part of each instruction specifies the operation to be done, and the second part specifies the memory location, or *address*, to be used

II. Assembly Languages (The Second Generation)

A. More "English-like," so somewhat easier for humans to use

B. Names (instead of numbers) are used to specify the operations to be done, and the memory locations involved

C. Typical equivalent assembly language code:

LOAD BASEPAY
ADD OVERPAY
STORE GROSSPAY

D. A translation program -- known as an *assembler* -- is used to translate each assembly language instruction into the comparable machine language instruction. The operation name (e.g., LOAD) is translated into the numeric code for the operation (e.g., 0101) and the *variable* name (e.g., BASEPAY) is associated with a particular numeric address in memory (e.g., 0011 1110 1000).

E. Since the assembly language of a given computer is based upon its machine language, each model computer has its own unique assembly language. So assembly language programs are still not portable.

F. Because each assembly language instruction is translated into exactly one machine language instruction, assembly language programs also tend to be quite lengthy.

III. High-Level Languages (The Third Generation)

A. Examples: FORTRAN, BASIC, COBOL, Pascal, Ada, C, C++, Java (many others)

B. Use English words and familiar symbols to specify operations, so easiest for humans to use

C. May be run on any computer that has a translation program -- a *compiler* (or, for some languages, an *interpreter*) -- that translates a program in a particular high-level language into the machine language of a particular computer.

For example, a C++ compiler on a Mac will translate C++ programs into the machine language of the Mac, while a C++ compiler on a PC will translate those same programs into the machine language of the PC. So high-level language programs *are* portable!

D. Further, since high-level languages are *not* based upon a particular machine language, a single high-level instruction may be translated into *several* machine language instructions.

So high-level language programs tend to be shorter than machine-language and assembly-language programs!

E. Advantages of high-level languages (summary)
1. Easiest to use
2. Programs are portable
3. Programs are shorter

F. Equivalent Java (or C++) code:

grossPay = basePay + overtimePay ;

WEB SOFTWARE

Web software is a collection of languages which include:

- Markup (e.g. XHTML)
- Scripting for dynamic content under which we have the
- Client side, using scripts embedded in the XHTML documents e.g.JavaScript, PHP
- Server side, using the common Gateway interface e.g. JSP, ASP,PHP
- General- purpose, executed on the web server through cGI e.g. Java, C++.

What are Programming Languages?

In order to communicate with others in our daily lives we rely on a common language (English). This language (along with other languages, Spanish, German, French, etc) has a common vocabulary, grammar, style, and definite set of rules. A particular language is also used to communicate with computers. These languages also possess a common vocabulary, grammar, style, and definite set of rules. Although computers appear to be very intelligent machines, they cannot think and reason on their own. Computers rely on humans to give them a set of instructions. These instructions are called programs. The language used to communicate these programs to the computer is called a programming language.
The set of rules of a particular programming language is called the language's syntax

History of Computer Programming Languages

Computer languages were first composed of a series of physical steps to wire a particular program. These languages progressed to a series of steps keyed into the computer and then executed. Finally, they have progressed to advanced features such as logical branching, iteration, modular programming, and object oriented programming.
Early "computers" and calculating devices could only be programmed by physical motion They were made to execute tasks by changing the gears which executed the calculations (1800's). Eventually, physical motion was replaced by electrical signals (1942, ENIAC). However, even the ENIAC could only be programmed by physically pre-setting switches and rewiring the entire system for each new program or calculation. Obviously, this process proved to be slow and prohibitive. In the mid-1940's a mathematician by the name of John Von Neumann made two critical improvements over the old program paradigm. **First,** he developed the technique that the actual computer hardware should be simple and multipurpose, not needing to be wired for each program. Instead, complex instructions should be used to control the simple hardware, allowing it to be reprogrammed much faster. **Secondly,** he developed the concept of "conditional control transfer." His idea stated that computer code could be branched or repeated with the logical statements such as IF or FOR. In addition, blocks of code could be repeated, which contributed to later concepts such as modular programming, libraries, and objected oriented programming. Within a computer, all data is represented by electronic switches that can be either off or on. An off switch is designated by a 0 and an on switch is designated by a 1. Because computers can understand only these on and off switches, the first programmers had to write the program instructions using nothing but combinations of 0's and 1's. Instructions written in 0's and 1's are called Machine Language

Machine language is considered a low-level language because it communicates with the computer at a very low or crude level. Slightly more advanced programming languages are called **Assembly Languages**

Assembly languages allow the programmer to use alphabetic mnemonics to replace the 0's and 1's in the program. For example, most assembly languages use the mnemonic ADD to represent an add operation. Programs written in assembly language require an assembler to convert the assembly instructions into machine code (0's and 1's). Assembly languages are considered mid-level languages because they communicate with the computer at a higher level than machine language.

By the late 1940's and early 1950's, high-level languages were first developed (languages which allow the programmer to use instructions that more closely resemble the English language. These languages all required a compiler or a interpreter to convert the English-like code into 0 's and 1's the computer can understand. In 1951, Grace Hopper wrote the first compiler.

In 1957, **FORTRAN (FORmula TRANslator),** the first of the major high-level languages was developed. This language was designed at IBM for scientific computing. Today, this language would be considered restrictive as it only included IF, DO, and GOTO statements. However, the basic types of data still in use today got their start in FORTRAN, these included logical variables (TRUE or FALSE), and integer, real, and double-precision numbers.

Business computing started taking off in 1959, and because of this, the language COBOL (Common Business-Oriented Language) was developed by Grace Hopper. It was designed from the ground up as the language for business. COBOL statements have a very English-like grammar, making it quite easy to learn. Versions of COBOL are still used by businesses today. In the 1960's, teaching languages were developed.

In 1964, the learning language BASIC (Beginner's All-Purpose Symbolic Instruction Code) was written by John Kemeny and Thomas Kurtz. FORTRAN and COBOL were improved into the language Pascal.

It was developed by Nicholas Wirth in the early 1970's. Its development was motivated by the desire to create a quality teaching language, however it proved to have many additional improvements. Pascal was the first language to use CASE statements and dynamic variables, had improved pointers, and used solid input/output and mathematical features. In 1972, C was developed by Dennis Ritchie while working at Bell Laboratories. The transition in usage from the first major languages to the major languages of today occurred with the transition between Pascal and C. C was built to be fast and powerful, at the expense of being somewhat cryptic. C is very commonly used to program operating systems such as Unix, Windows, the Mac operating system, and Linux. In the 1980's, the concept of Object Oriented Programming was developed. Objects could be defined (data types or other objects) with a specific set of characteristics and rules. These objects could be manipulated, changed, and used within other programs. Bjarne Stroustrup supported this concept in his development of the language C++ " (C with classes). C++ is most often used in simulations, such as games. In the 1990's, technology dictated that computer programming languages become more portable (can run on many type of machines). Sun Microsystems developed the programming language Java to meet this need. Also, Java uses the concept of Object Oriented programming extensively throughout the language. Java is said to be a textbook example of a good programming language of today.

Visual Basic was developed by Microsoft in 1991. With the development of the Microsoft Windows graphical user interface in the late 1980's and the early 1990's, BASIC evolved into Visual Basic. Visual Basic greatly simplifies Windows applications development.

CRITERIA FOR LANGUAGE EVALUATION AND COMPARISION

1. Expressivity
2. Well – Definiteness
3. Data types and structures
4. Modularity
5. Input-Output facilities
6. Portability
7. Efficiency
8. Pedagogy
9. Generality

Expressivity means the ability of a language to clearly reflect the meaning intended by the algorithm designer (the programmer). Thus an "expressive" language permits an utterance to be compactly stated, and encourages the use of statement forms associated with structured programming (usually "while "loops and "if – then – else" statements).

By **"well-definiteness"**, we mean that the language's syntax and semantics are free of ambiguity, are internally consistent and complete. Thus the implementer of a well defined language should have, within its definition a complete specification of all the language's expressive forms and their meanings. The programmer, by the same virtue should be able to predict exactly the behavior of each expression before it is actually executed.

By **"Data types and Structures",** we mean the ability of a language to support a variety of data values (integers, real, strings, pointers etc.) and non elementary collect ions of these.

Modularity has two aspects: the language's support for sub-programming and the language's extensibility in the sense of allowing programmer – defined operators and data types. By **sub programming**, we mean the ability to define independent procedures and functions (subprograms), and communicate via parameters or global variables with the invoking program.

In evaluating a language's **"Input-Output facilities"** we are looking at its support for sequential, indexed, and random access files, as well as its support for database and information retrieval functions.

A language which has **"portability"** is one which is implemented on a variety of computers. That is, its design is relatively "machine – independent". Languages which are well- defined tend to be more portable than others.

An **"Efficient"** language is one which permits fast compilation and execution on the machines where it is implemented. Traditionally, FORTRAN and COBOL have been relatively efficient languages in their respective application areas.

Some languages have better **"Pedagogy"** than others. That is, they are intrinsically easier to teach and to learn, they have better textbooks; they are implemented in a

better program development environment, they are widely known and used by the best programmers in an application area.

Generality: Means that a language is useful in a wide range of programming applications. For instance, APL has been used in mathematical applications involving matrix algebra and in business applications as well.